Importar los datos

In [40]:
```
import pandas as pd
```

In [41]:
```
# Importar los datos
df = pd.read_csv("countries.csv")
df.head()
```

Out[41]:

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)	Phones (per 1000)	Arable (%)	Crops (%)	Other (%)	Climate	Birth
0	Afghanistan	ASIA (EX. NEAR EAST)	31056997	647500	48.0	0.00	23.06	163.07	700.0	36.0	3.2	12.13	0.22	87.65	1.0	4
1	Albania	EASTERN EUROPE	3581655	28748	124.6	1.26	-4.93	21.52	4500.0	86.5	71.2	21.09	4.42	74.49	3.0	1
2	Algeria	NORTHERN AFRICA	32930091	2381740	13.8	0.04	-0.39	31.00	6000.0	70.0	78.1	3.22	0.25	96.53	1.0	1
3	American Samoa	OCEANIA	57794	199	290.4	58.29	-20.71	9.27	8000.0	97.0	259.5	10.00	15.00	75.00	2.0	2
4	Andorra	WESTERN EUROPE	71201	468	152.1	0.00	6.60	4.05	19000.0	100.0	497.2	2.22	0.00	97.78	3.0	

In [42]:
```
# Eliminar la variable (columna) "Region"
df = df.drop(columns=["Region"]) if "Region" in df.columns else df

# Eliminar missing values
df = df.dropna()

# Comprobar que no existe la columna "Region"
print("No existe la columna 'Region'") if "Region" not in df.columns else print("ERROR: La columna 'Region' sigue en el dataframe")

# Comprobar que no quedan missing values
# df.isna().sum() hace la suma por columnas, devuelve los missing values de cada columna
print("No hay ningún valor faltante") if df.isna().sum().sum() == 0 else print("ERROR: Aún hay valores faltantes")

# Ver cantidad filas del dataframe
print("Dimensiones del dataframe: ", df.shape)

# Ver dataframe
df.head()
```

```
No existe la columna 'Region'
No hay ningún valor faltante
Dimensiones del dataframe:  (179, 19)
```

Out[42]:

	Country	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)	Phones (per 1000)	Arable (%)	Crops (%)	Other (%)	Climate	Birthrate	Death
0	Afghanistan	31056997	647500	48.0	0.00	23.06	163.07	700.0	36.0	3.2	12.13	0.22	87.65	1.0	46.60	2
1	Albania	3581655	28748	124.6	1.26	-4.93	21.52	4500.0	86.5	71.2	21.09	4.42	74.49	3.0	15.11	
2	Algeria	32930091	2381740	13.8	0.04	-0.39	31.00	6000.0	70.0	78.1	3.22	0.25	96.53	1.0	17.14	
6	Anguilla	13477	102	132.1	59.80	10.76	21.03	8600.0	95.0	460.0	0.00	0.00	100.00	2.0	14.17	
7	Antigua & Barbuda	69108	443	156.0	34.54	-6.15	19.46	11000.0	89.0	549.9	18.18	4.55	77.27	2.0	16.93	

Generar los datos de entrenamiento:

X serán todas las variables (columnas) salvo GDP. Y será la variable a predecir, en este caso, la columna GDP.

In [43]:
```
# Variables predictoras (todas menos GDP)
X = df.drop(columns=["GDP ($ per capita)"])

# Variable objetivo
y = df["GDP ($ per capita)"]

# Comprobación de X
print("Variables predictoras X:", X.columns)

# Comprobación de y
print("\nVariable objetivo y: ", y.head())

# Comprobar que ambas tienen las mismas observaciones
print(f"\nObservaciones en X: {len(X)}, Observaciones en y: {len(y)}")
```

```
Variables predictoras X: Index(['Country', 'Population', 'Area (sq. mi.)', 'Pop. Density (per sq. mi.)',
       'Coastline (coast/area ratio)', 'Net migration',
       'Infant mortality (per 1000 births)', 'Literacy (%)',
       'Phones (per 1000)', 'Arable (%)', 'Crops (%)', 'Other (%)', 'Climate',
       'Birthrate', 'Deathrate', 'Agriculture', 'Industry', 'Service'],
      dtype='object')

Variable objetivo y:  0      700.0
1     4500.0
2     6000.0
6     8600.0
7    11000.0
Name: GDP ($ per capita), dtype: float64

Observaciones en X: 179, Observaciones en y: 179
```

Normalizado o estandarizado:

Tal y como se ha visto en la asignatura, normalizar o estandarizar las variables es fundamental para diseñar e implementar un perceptrón multicapa debido al uso del descenso por gradiente. Si las variables pueden tomar valores muy distintos entre sí y no se estandarizan, los gradientes pueden quedar desbalanceados, lo que puede provocar que el modelo se quede atascado en mínimos locales, no converja o aumente considerablemente el tiempo de entrenamiento. Por ejemplo, variables con valores muy grandes generan gradientes enormes, mientras que variables con valores pequeños producen gradientes diminutos. Esto hace que algunos pesos se ajusten en exceso (correcciones demasiado grandes, dificultando la convergencia) y otros apenas se actualicen (ralentizando el aprendizaje).

En el caso del dataframe en estudio, se observa que la variable "GDP" varía en un rango de 10.000 unidades, generando gradientes muy grandes, mientras que "Literacy" en una escala de 100, generando gradientes muy pequeños. Consecuentemente, es necesario estandarizar o normalizar.

Se ha optado por StandardScaler sobre MinMaxScaler porque no todas las variables observadas tienen un rango fijo y conocido, por ejemplo, "GPD" podrá ser cualquier valor a partir de 0, y MinMaxScaler, al transformar los datos al rango [0, 1], puede distorsionar la escala relativa entre observaciones cuando no existe un límite superior natural, haciendo que el modelo sea más sensible a valores extremos. En cambio, StandardScaler centra y escala las variables según su distribución, proporcionando gradientes más equilibrados y una convergencia más estable, favoreciendo además la incorporación de nuevas observaciones 'online'.

Las operaciones se hacen sobre el dataframe X, que son las variables predictoras que utiliza el modelo para aprender.

Convertir las variables categoriales a numéricas:

En este caso, la variable "Country" resulta interesante para la tarea, por lo que hay que convertirla a numérica. Para ello, se convertirá el dataframe a una matriz esparsa (one-hot encoding), puesto que si se asigna, por ejemplo, un identificador numérico a cada país (1, 2, 3, ...), se introduce un falso orden entre países que el modelo podría aprender.

Las operaciones se hacen sobre el dataframe X, que son las variables predictoras que utiliza el modelo para aprender.

In [44]:
```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Identificar columnas
cat_cols = ["Country"]
num_cols = [c for c in X.columns if c != "Country"]

# Preprocesado: estandarización y cat2num
preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ]
)
```

Creación de los modelos con validación cruzada:

Los modelos se crean mediante pipelines que incluyen la estandarización de los datos (StandardScaler). Los pipelines son cadenas de pasos encadenados que se ejecutan siempre en el mismo orden: primero el preprocesado, luego el modelo, etc...

In [48]:
```python
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, KFold
import numpy as np
```

In [61]:
```python
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline

# MLP1: 1 capa oculta de 200 neuronas
mlp1 = Pipeline([
    ("preprocess", preprocess),
    ("mlp", MLPRegressor(hidden_layer_sizes=(200,),
                         alpha=0.1,
                         max_iter=3000,
                         random_state=42))
])

# MLP2: 2 capas ocultas de 50 neuronas
mlp2 = Pipeline([
    ("preprocess", preprocess),
    ("mlp", MLPRegressor(hidden_layer_sizes=(50, 50),
                         alpha=0.1,
                         max_iter=3000,
                         random_state=42))
])
```

In [62]:
```python
kf = KFold(n_splits=5, shuffle=True, random_state=42)

scores_mlp1 = cross_val_score(mlp1, X, y, cv=kf, scoring="neg_mean_squared_error")
scores_mlp2 = cross_val_score(mlp2, X, y, cv=kf, scoring="neg_mean_squared_error")
```

```python
print("MLP1 MSE medio:", -np.mean(scores_mlp1))
print("MLP2 MSE medio:", -np.mean(scores_mlp2))
```

```
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
MLP1 MSE medio: 39105554.78433666
MLP2 MSE medio: 19300925.701751813
/home/javierac/anaconda3/envs/sistemas_inteligentes/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:78
1: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (3000) reached and the optimization hasn't converged yet.
  warnings.warn(
```

¿Alguno de los dos perceptrones domina al otro? ¿Por qué crees que se producen las diferencias?

Claramente el segundo perceptrón multicapa, MLP2, domina al primero, puesto que su error medio es prácticamente la mitad. Esto está completamente alineado con la teoría estudiada en la asignatura, ya que las capas ocultas permiten al modelo aprender relaciones no lineales y combinaciones complejas entre las variables de entrada. En este caso, MLP2, al disponer de dos capas ocultas, puede construir representaciones más profundas y jerárquicas de los datos, capturando patrones que un modelo con una sola capa no puede modelar adecuadamente.

Además, en este problema se incluye una variable categórica (el país), que ha sido transformada mediante One-Hot Encoding. Esta codificación genera un conjunto de variables binarias que representan la pertenencia a cada país, lo que introduce un espacio de entrada de mayor dimensionalidad y con interacciones más complejas. Una red más profunda, como MLP2, es capaz de combinar estas variables categóricas con las numéricas de forma más eficaz, aprendiendo diferencias estructurales entre países que influyen en el PIB.

Por el contrario, MLP1, al tener una única capa oculta muy grande, tiende a comportarse como un modelo menos profundo y más propenso al sobreajuste, ya que al no ser capaz de identificar patrones fuertemente no lineales intenta memorizarlos, lo que le impide generaliza correctamente. Esto explica que su rendimiento en validación cruzada sea peor: no logra capturar adecuadamente la no linealidad del problema y su capacidad de generalización es inferior.